

OHM - Oral History Manager

OHM is a desktop application built with Flet that streamlines the creation and processing of oral history recordings for Digital.Grinnell. The application provides an integrated workflow from audio conversion through transcription to final deliverables.

Features

Function 0: Merge Audio Files (Optional)

- Concatenates multiple WAV or MP3 files into a single recording
- Useful when recordings are split across multiple files
- Preserves file order and quality
- Automatically archives source files after merge
- Creates merge provenance JSON sidecar file
- Requires FFmpeg

Function 1: Convert WAV to MP3

- Converts large WAV audio files to compressed MP3 format
- Uses FFmpeg for high-quality conversion
- Preserves audio quality while reducing file size
- Required for files that need to be uploaded to transcription services

Function 2: Transcribe with MS Word Online

- **Cloud-based** Microsoft transcription service
- **Requires Microsoft 365 subscription**
- Uses Word's built-in Transcribe feature
- Provides speaker identification
- Includes **automated DOCX to JSON conversion** with button click
- Automatically maps Word speaker labels to your custom speaker names
- Dialog has two tabs:  **MS Word Instructions** and  **Review Notes** — take notes without leaving the transcription dialog

Note on Whisper: OpenAI's Whisper transcription was evaluated and implemented as an automated local transcription option but ultimately removed from OHM due to licensing concerns with the model and lack of built-in speaker diarization capabilities. MS Word Online, while requiring a subscription, provides superior speaker identification and meets the needs of the oral history workflow.

Function 3: Edit Review Notes

- Opens an in-app Markdown editor for the selected oral history
- Creates `review_notes.md` in the file's output directory if it doesn't exist
- Seeds new files with a template (title, filename, date, blank Notes section)
- **Save** writes the file and closes the dialog; write errors are shown inline

- **Cancel** closes without saving any changes
- File lives alongside all other per-file outputs in `<sanitized-basename>--dg_<epoch>/`
- Also accessible as the  **Review Notes** tab inside the Function 2 dialog — edits made there are shared with this function

Function 4: Generate TXT, VTT, CSV & PDF from JSON

- Reads edited JSON transcript (from Function 2)
- Generates four output formats:
 - **TXT** - Plain text with timestamps and speaker labels
 - **VTT** - WebVTT subtitle format for video players
 - **CSV** - Spreadsheet-friendly format with timestamp, speaker, and words columns
 - **PDF** - Formatted document with descriptive title, provenance section, and transcript
- All outputs include:
 - Audio file header
 - Transcript section
 - Timestamps in [HH:MM:SS] format
 - Speaker names
 - Professional formatting

Function 5: Report Workflow Progress

- Scans input directory and OHM-data for status comparison
- Generates timestamped markdown reports
- Tracks completion across all workflow stages
- Identifies:
 - Complete files (all stages finished)
 - In-progress files (partial completion)
 - Not started files (source only)
- Shows statistics and percentages
- Helps track project progress over time

Additional Features

Working/Output Directory

- **User-selectable** output location separate from the input directory
- Defaults to the same directory chosen as the Input Directory
- An **OHM-data** subfolder is automatically created inside the chosen directory
- Changing the Input Directory auto-updates the working/output directory unless it has been manually set
- Selection persists across sessions

Help Mode

- **Built-in help documentation** for each function
- Click Help Mode checkbox before selecting a function
- Displays detailed instructions in scrollable dialogs
- Context-sensitive help based on transcription mode

Setup

Running from Source (developers / contributors)

1. Clone the repository

```
git clone <repository-url>  
cd OHM
```

2. Run the application

```
./run.sh
```

This script will:

- Create a Python virtual environment (`.venv`)
- Install all dependencies from `python_requirements.txt`
- Launch the OHM application

Installing from DMG (end users)

See [Distribution & DMG](#) below for pre-built installer instructions.

Requirements

System Requirements

- **macOS** (primary platform)
- **Python 3.8+**
- **FFmpeg** (for WAV to MP3 conversion)

```
brew install ffmpeg
```

Python Dependencies

All dependencies are installed automatically by `run.sh`:

- `flet==0.25.2` - Desktop UI framework
- `python-docx` - DOCX file parsing
- `reportlab` - PDF generation

Optional Requirements

- **Microsoft 365 subscription** (only for MS Word Online transcription mode)

Workflow

Complete Oral History Processing Workflow

0. Merge Audio Files (optional)

- If your recording is split across multiple files, run **Function 0** first
- Select files in the correct order and merge them
- Source files are automatically moved to a **Merged/** subdirectory
- The merged file becomes your new source file

1. Select Input Directory

- Browse to folder containing WAV or MP3 audio files
- Click "List WAV and MP3 Files" to scan
- Select an audio file from the dropdown

1a. Set Working/Output Directory (optional)

- Defaults to the same directory as the Input Directory
- An **OHM-data** subfolder is created there automatically
- Override with Browse... to write outputs to a different location (e.g., an external drive)
- The setting persists across sessions

2. Convert Audio (if needed)

- If you have WAV files, run **Function 1** to convert to MP3
- MP3s are required for cloud transcription services
- Skip this step if you already have MP3 files

3. Choose Transcription Method

- Use **Function 2** for MS Word Online transcription

4. Transcribe Audio

- Run **Function 2** to create initial transcript
- Follow MS Word Online instructions, download DOCX, click "Convert to JSON"

5. Edit Transcript JSON

- Open the generated **dg_<epoch>_transcript.json** file
- Fix speaker labels (e.g., change "SPEAKER_00" to actual names)
- Correct transcription errors
- Adjust timing if necessary
- Save the edited JSON

6. Generate Final Outputs

- Run **Function 4** to create deliverables
- Generates: TXT, VTT, CSV, and PDF files
- All formats include timestamps and speaker labels
- PDF opens with a descriptive title and provenance section

7. Add Review Notes (Optional)

- Run **Function 3** to open the Markdown review notes editor
- Records observations, corrections, or follow-up items for the oral history
- Saved as **review_notes.md** in the file's output directory

8. Track Progress (Optional)

- Run **Function 5** to generate progress report
- Review what's complete and what needs work
- Reports saved with timestamps in the working/output directory

Output Structure

All processed files are organized inside an **OHM-data** subfolder within the **Working/Output Directory** you select (defaults to the Input Directory):

```
<working_dir>/OHM-data/
├── <sanitized-basename>--dg_<epoch>/
│   ├── dg_<epoch>.wav          # Original or converted audio
│   ├── dg_<epoch>.mp3          # Compressed audio
│   ├── dg_<epoch>_transcript.json # Editable transcript
│   ├── dg_<epoch>.txt          # Plain text output
│   ├── dg_<epoch>.vtt          # Video subtitle format
│   ├── dg_<epoch>.csv          # Spreadsheet-friendly output
│   ├── dg_<epoch>.pdf          # Formatted document with provenance
│   └── review_notes.md         # Reviewer notes (Function 3)
└── workflow_progress_YYYYMMDD_HHMMSS.md # Progress reports
```

Naming convention: The basename part is sanitised (spaces → **_**, special characters stripped). For merged audio files the suffix **_MERGED** is appended before the **--dg_** separator, e.g. **Kerry_Bart_MERGED--dg_1775659390/**.

Note: Application logs and the persistent settings file remain in **~/OHM-data/** regardless of the Working/Output Directory selection.

Migrating existing OHM-data directories

If you have OHM-data directories created by an older version of OHM (which used a **- dg_** separator or lowercase **_merged**), run the included migration script to bring them up to date:

```
# Preview changes (no files are renamed):
python3 migrate_ohm_names.py /path/to/OHM-data

# Apply all renames:
python3 migrate_ohm_names.py --apply /path/to/OHM-data
```

Migrating existing OHM-data directories

If you have OHM-data directories created by an older version of OHM (which used a `- dg_` separator or lowercase `_merged`), run the included migration script to bring them up to date:

```
# Preview changes (no files are renamed):
python3 migrate_ohm_names.py /path/to/OHM-data

# Apply all renames:
python3 migrate_ohm_names.py --apply /path/to/OHM-data
```

File Formats

JSON Transcript

```
{
  "notes": {
    "narrative": "Human-readable provenance paragraph ...",
    "created_at": "2026-04-07 14:24:13",
    "transcription_method": "MS Word Online (manual transcription)",
    "app": "OHM - Oral History Manager",
    "system": { "hostname": "...", "os_name": "Darwin", "machine": "arm64" },
    "speaker_mapping": {
      "Interviewer": "Interviewer",
      "Speaker 1": "Jane Smith",
      "Reviewed By": "Mark McFate"
    },
    "source_audio": { "...": "technical metadata" }
  },
  "language": "en",
  "segments": [
    {
      "start": 0.0,
      "end": 5.42,
      "text": "Hello, welcome to the interview.",
      "speaker": "Interviewer"
    }
  ]
}
```

TXT/PDF Format

Audio file
dg_1775499960.mp3

Transcript

[00:00:00] John Doe
Hello, welcome to the interview.

```
[00:00:05] Jane Smith  
Thank you for having me.
```

Help Documentation

Each function has detailed help documentation accessible via Help Mode:

- **FUNCTION_0_MERGE_AUDIO.md** - Audio merging guide (optional preprocessing)
- **FUNCTION_1_WAV_TO_MP3.md** - Audio conversion guide
- **FUNCTION_2_MS_WORD_ONLINE.md** - MS Word Online transcription guide
- **FUNCTION_3_REVIEW_NOTES.md** - Review notes editor guide
- **FUNCTION_4_GENERATE_OUTPUTS.md** - Output generation guide
- **FUNCTION_5_REPORT_PROGRESS.md** - Progress reporting guide

Privacy & Local Processing

- **Functions 0, 1, 4, and 5** run entirely locally
- **Function 3** also runs entirely locally (Markdown editor)
- **No audio or text is uploaded to external servers** except via MS Word Online (Function 2)
- **Function 2 (MS Word Online)** uses Microsoft's cloud service
- All processing preserves original files
- Speaker names and settings stored locally in JSON

Troubleshooting

FFmpeg Not Found

```
brew install ffmpeg
```

MS Word Transcription Issues

- Ensure you have an active Microsoft 365 subscription
- Word Online transcription requires < 200MB audio files
- Use Function 1 to convert/compress large files first
- Microsoft imposes a **300-minute-per-month** transcription limit; if exceeded, the Transcribe pane will be unavailable until the limit resets

PDF Generation Fails

- Ensure **reportlab** is installed: `pip install reportlab`
- Check that output directory exists and is writable

Distribution & DMG

Prerequisites for recipients

Recipients need these installed once before using OHM:

- **Python 3** — python.org/downloads or `brew install python`
- **FFmpeg** — `brew install ffmpeg`
- **Homebrew** (recommended) — brew.sh

Building the DMG

Run the build script from the project root. An optional version argument defaults to **1.0**:

```
bash build_dmg.sh          # produces OHM_v1.0.dmg
bash build_dmg.sh 1.2      # produces OHM_v1.2.dmg
```

The script:

1. Creates a macOS **.app** bundle (**OHM.app**) with a shell launcher
2. Bundles all source files into **OHM.app/Contents/Resources/src/**
3. Excludes **.venv**, **.git**, **.env**, log files, and any existing DMGs
4. Compresses everything into **OHM_v<version>.dmg** using the built-in **hdiutil** — no extra tools required

The resulting DMG is ~80 KB. Dependency installation happens on the recipient's machine at first launch.

Installing from the DMG

1. Open **OHM_v<version>.dmg**
2. Drag **OHM.app** to your Applications folder (or any convenient location)
3. Eject the DMG

First launch (Gatekeeper)

Because the app is not code-signed, macOS Gatekeeper will block a plain double-click the first time:

1. **Right-click** **OHM.app** → **Open**
2. Click **Open** in the confirmation dialog
3. Subsequent launches work with a normal double-click

Alternative: System Settings → Privacy & Security → scroll to the blocked-app notice → click **Open Anyway**

What happens on first launch

A Terminal window opens and automatically:

- Creates a Python virtual environment inside the app bundle
- Installs all Python dependencies (may take a few minutes on first run)
- Launches the OHM window when setup is complete

The Terminal window can be left open or minimised; closing it will also close OHM.

Subsequent launches

Dependencies are cached in the virtual environment; startup is fast after the first run.

Notes

- Generated DMG files are excluded from version control via [.gitignore](#)
 - The app's [src/](#) directory ([OHM.app/Contents/Resources/src/](#)) can be opened in Finder to access or edit source files directly
-

Development

The application uses:

- **Flet** for the desktop UI framework
- **python-docx** for DOCX file parsing
- **reportlab** for PDF generation
- **Persistent storage** via JSON for settings and usage tracking

Log Files

Application logs are stored in:

```
~/OHM-data/logfiles/ohm_YYYYMMDD_HHMMSS.log
```

License

[License information to be added]

Contributing

[Contribution guidelines to be added]

Support

For questions or issues, please refer to the function-specific help documentation or contact the Digital.Grinnell team.

Changelog

For a complete history of changes, new features, and bug fixes, see [CHANGELOG.md](#).